Stuart School of Business

ILLINOIS INSTITUTE OF TECHNOLOGY

MAX:506
Database Design & SQL
Professor- Dr. Dinakar Jayarajan

DBMS Project Report
FOXCORE RETAIL (A): DESIGNING A DATABASE

Group 1

1. Akanksha Shrivastava
2. Anandi Singh
3. Hetal Virendrabhai Gandhi
4. Unnati Agrawal
5. Jahnvi Vishal Kansara
6. Diana Nyaama Asilga

Sr. No	Topic	Page No
1	Index	2
2	Introduction	3
3	Problem Statement	3
4	Objective	3
5	Methodology	4 - 5
6	Key Business Entities and Functional Scope	6
7	Database Architecture view, Key Business Constraints, System Requirements and Performance Expectations	7
8	Database Design and Implementation	8
9	Relational Database Schema	9
10	DDL Statements – Create Commands	10 - 14
11	DDL Statements - Insert / Update / Delete Commands	15 - 17
12	DDL Statements - Select Queries	18 – 23
13	Conclusion	24

Introduction :

Foxcore Retail is a rapidly expanding startup based in Ontario, co-founded by Mitchell Fox and Liam Corrigan.

The company specializes in selling novelty products at music festivals and trade show events, like top-selling bubble guns and Arctic cooling towels. As Foxcore expanded, it became very challenging to keep track of sales transactions, employee schedules, and inventory updates using manual, spreadsheet based systems.

This project proposes a formal relational database system to enable improved efficiency, decision support, and future development support through effective tracking of important business activities. It also aims to improve efficiency, decision making and position of the company for its expansion.

Problem Statement:

Foxcore's sales tracking was managed manually, which subsequently entered into Excel, leading to significant administrative burdens, data inaccuracies and redundancy. These inefficiencies were particularly evident in the calculation of sales commissions and the inability to leverage sales data for strategic insights.

As the business expanded, the need for an effective database system to record the increasing amounts of data across multiple venues and events became critical. Due to incomplete and outdated records the Inventory estimates were inaccurate leading to missed sales opportunities or excess stock. With operations expanding it becomes very difficult to implement a centralized, reliable and scalable database system to manage volume of business information.

Objective:

- Eliminates redundancy and data inconsistency.
- To get a real time track of the product sales, employee commissions, and event performance.
- Enhances planning and operational decision-making.
- Facilitates performance monitoring across products, sales consultants, and event venues.
- Consolidated and centralized Business data.
- Improve inventory management.
- Streamline administrative processes.
- Support scalability.

Methodology:

Designing a database for Foxcore Retail involves a structured methodology that ensures the database is scalable, efficient, and capable of handling the business's operational and analytical needs. Here's a step-by-step methodology for database design that Foxcore Retail can employ:

Step 1: Conceptual database design

- Identify entity types: Identify all real world entities the system needs (e.g., Product, Sale, Event, Salesperson, Booth).
- Identify relationship types: Defining how entities are related to one another (e.g., Salesperson sells products at Booths during Events).
- Identify and associate attributes with entity or relationship types: Characterise each entity (e.g., ProductName, WholeSaleCost for Product entity).
- Determine attribute domains: Specify valid data types and possible values (e.g., PhoneNumber as VARCHAR, StartDate as DATE).
- Determine candidate, primary, and alternate key attributes: Make sure each table can be uniquely defined (e.g., SaleID as Primary Key).
- Check model for redundancy: Make sure there are no unnecessary duplicates among entities.
- Validate conceptual model against user transactions: Verifying completeness via simulating real-world business processes (e.g., recording a sale).
- Review conceptual data model with user: Validating understanding and requirements by conducting sessions with company's stakeholders.

Step 2: Logical Database Design - Build and validate logical data model.

- Derive relations for logical data models: Turning entities and relationships into relational tables.
- Validate relations using normalization: Ensuring tables to achieve at least a 3NF for minimizing redundancy.
- Validate relations against user transactions: Checking if all business operations can be supported (e.g., commission calculation)
- Check integrity constraints: Explaining Primary Keys, Foreign Keys, unique constraints and not null constraints.
- Review logical data models with users: Sharing the schema draft for reviewing.
- Merge local logical data models into global models: If needed, integrating sub-models to a unified database structure.

Step 3: Physical Database Design - Translate logical data model for target DBMS.

- Design base relations: Executing tables and their attributes into their DBMS (e.g., SQL tables for Venue, Event).
- Design representation of derived data: Schedule calculated fields (such as total sales revenue) via views.
- Design general constraints: Executing integrity rules at the database levels.

Step 4: Physical Database Design - Design file organizations and indexes

- Analyze transactions: Recognize typical operations (e.g., heavy sales insertions, commission queries).
- Choose file organization: Opt for heap, clustered, or indexed file structures.
- Choose indexes: Create indexes in fields which are commonly searched.
- Estimate disk space requirements: Ensuring growth of project data to hardware scalability.

Step 5: Design user views: Improving specific views for different user roles (e.g., Manager View, Salesperson View, Admin Dashboard) to clarify access and enhance security.

Step 6: Design security mechanisms: Use role-based access control:

- Salespersons: Few access to their own sales records.
- Managers: Availability to commission, sales summaries, and event performance.Admins:
- Complete control including database backups, configuration, and audits.

Step 7: Consider the introduction of controlled redundancy: Selective duplicating of non-sensitive information where performance bottlenecks occur and takes the most time (i.e. stored "total sales").

Step 8: Monitor and tune the operational system: Implement monitoring to observe database performance and improve slow queries, indexes, and storage usage progressively over time.

Key Business Entities and Functional Scope :

Entity	Description	Occurrences
Venue	Physical location where events are held, includes name, address, and setup notes.	Each venue can host multiple events over time.
Event	Represents individual sales opportunities such as music festivals or trade shows.	Each event occurs at a specific venue and within specific dates.
Booth	Designated retail space at an event, assigned by location and size.	Each event has at least one booth; larger events may have multiple booths.
Product	Items available for sale with cost, price, description, and category data.	Products can be sold across multiple events and booths.
Inventory	Tracks stock levels, restocking dates, and reorder thresholds for each product.	Each product has a real-time inventory record that updates after each sale.
Salesperson	Individuals hired to sell products, with personal and employment data tracked.	A salesperson can work multiple shifts, but only one per time slot.
Salary	Stores base salary information and effective date for payroll reporting.	One record per change in salary per salesperson.
Shift	A working session assigned to a salesperson and booth, with time and role data.	Each booth can have multiple shifts across different time slots.
Sale	Records transactions including quantity, sale price, product, and payment method.	Each sale is linked to a shift, product, booth, event, and salesperson.
Commission	Calculates earnings on each sale based on a rate and sale value.	Each sale can generate one commission entry per salesperson.
EventPerformance	Aggregates feedback, customer count, and total revenue for post-event review.	One per event to evaluate overall success and customer experience.

Database Architecture view: Core features

- Many-to-One: Each booth is tied to a specific event; each sale is linked to a single salesperson and product.
- One-to-Many: Events have multiple booths; products are sold across multiple events.
- Cross-Linking: Sales link to events, products, salespersons, and shifts to support detailed analysis.
- Inventory Management: Current stock, restock dates, and reorder levels are tracked per product.
- Many-to-Many:Products are sold through many sales, and every sale must involve at least one product.

Key Business Constraints:

- Salespersons may have multiple shifts but never simultaneously.
- Each sale is linked to a valid product, salesperson, and booth within an open event.
- Inventory quantities must permit real-time modification versus sales activity.
- Role-based access control is applied.
- Salespersons: Read-only to their shift and sales information.
- Managers: To performance metrics, commission tracking, inventory, and event details.
- Admins: Full access to data models, backup processes, and analytics.

System Requirements and Performance Expectations:

Component	Initial Requirement	Scalability
Events	5+ annually	Dynamic addition, multi-venue support
Products	28 core SKUs	Expandable inventory and seasonal SKUs
Salespersons	17 active	Flexible addition/removal per season
Sales Records	10+ invoices (historical)	Real-time logging during events
Commission Processing	Per-shift, per-sale	Calculated via dynamic SQL views

Database Design and Implementation:

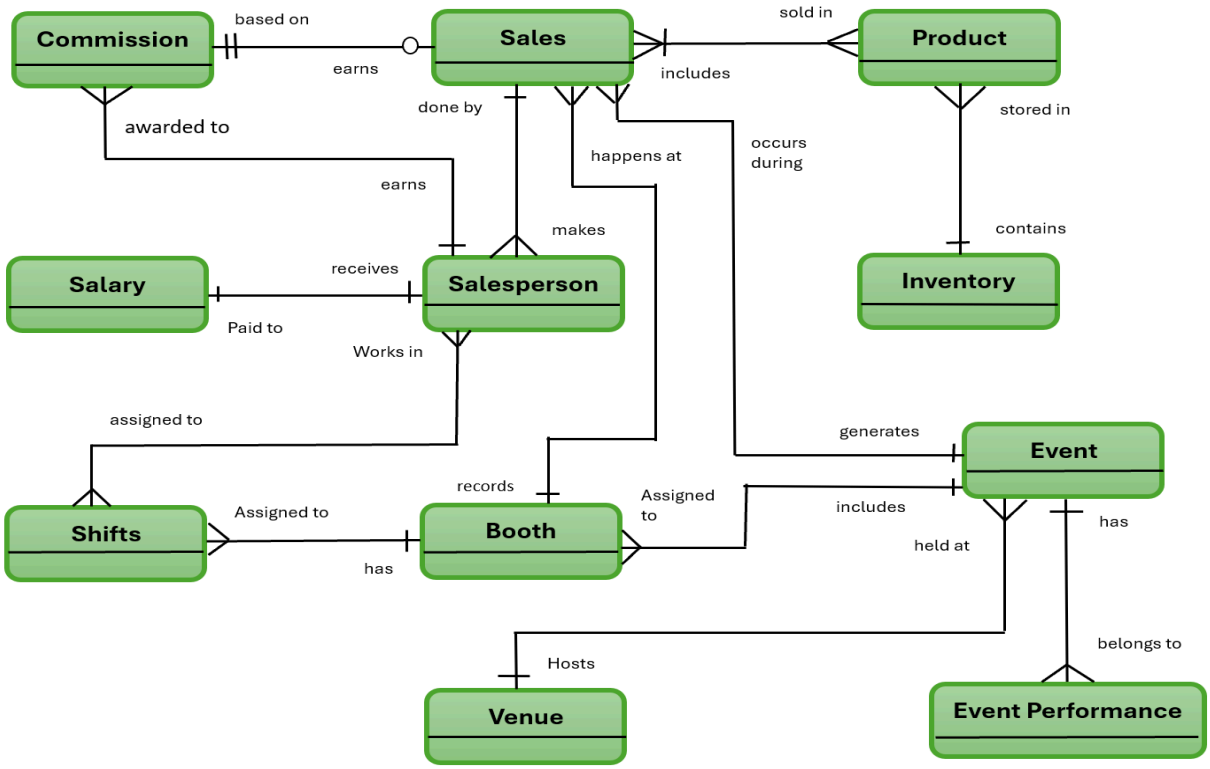
The newly revised database schema has been carefully crafted to capture and process key business data in twelve interrelated categories(Entities): Venues,Events, Booths, Products, Inventory, Salespersons, Salaries, Shifts, Sales, Commissions, and EventPerformance.

This integrated system enables end-to-end monitoring of Foxcore's business processes right from venue selection and event reservations to inventory management, shift allocation, sales capture in real time, and commission processing. All the parties are relationally inter-connected to enable below:

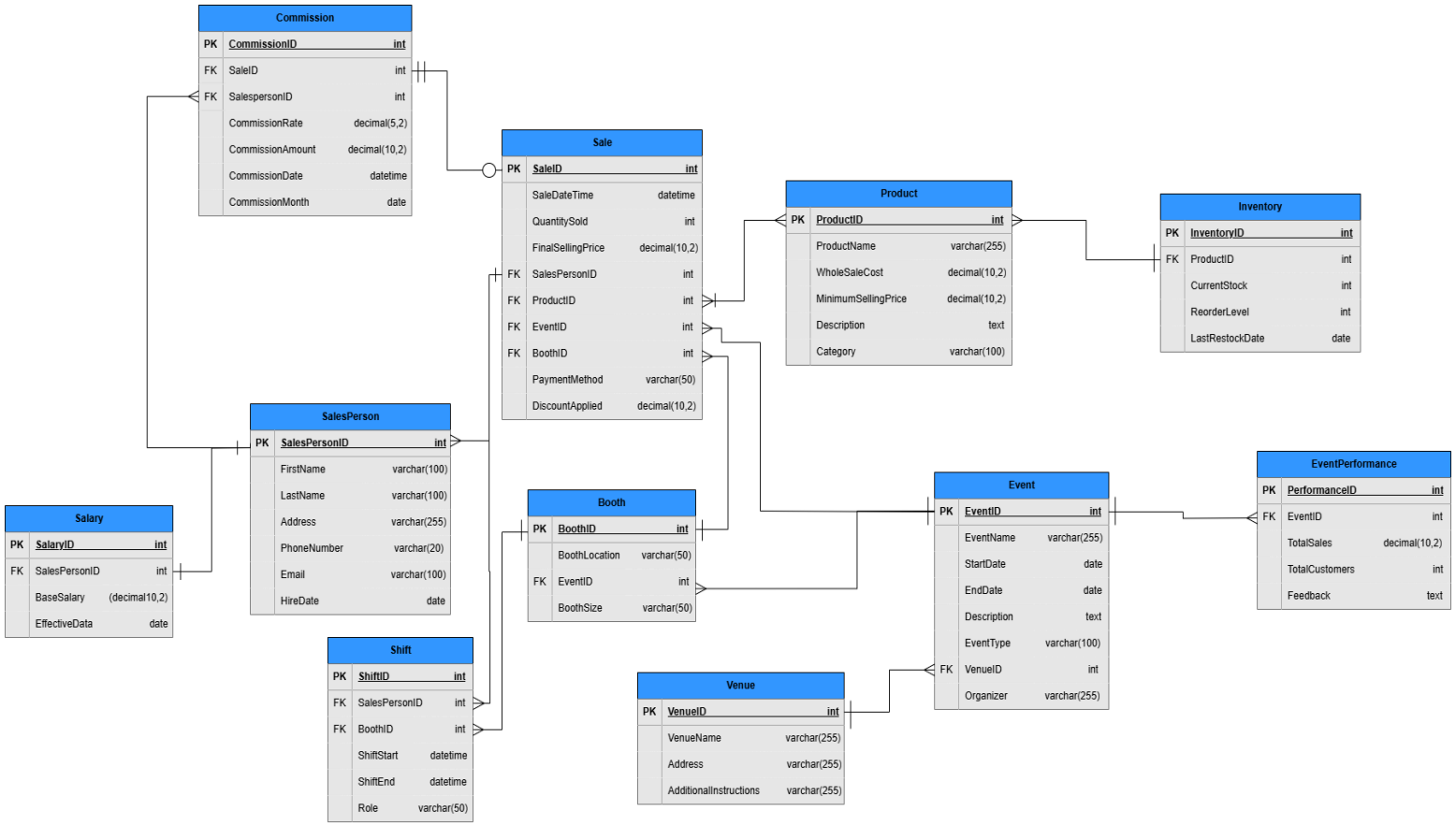
- Efficient entry and retrieval of information across different business domains.
- Real-time updates for product availability, booth performance, and event profits.
- Automated calculation of commission and salary disbursement.
- Scalability to allow future expansion of product lines, staff, and attendee numbers at events.

The database is fully normalized to eliminate redundancy and preserve data integrity, and referential constraints are imposed on all foreign keys. The structure also provides space for integration with analytical views and reporting products to support strategic business intelligence.

E-R Model Diagram for Foxcore Retail



Relational Database Schema:



Initial Database Size:

- **Events**: Beginning with a single music festival in the first year and scaling up to three events per weekend by the second year.
- **Venues**: Covering diverse geographic regions and event types with multiple unique locations.
- **Booths**: Several booths per event, each linked to individual sales shifts and optimized for crowd distribution.
- **Products**: Initially launching with bubble guns, the catalog expands to 28 diverse products within three years.
- **Shifts**: Average of four shifts for each two-day event, scalable based on event scope.
- **Salespersons**: Growth from a lean team to 17 active salespeople within three years.
- **Sales Transactions**: Exponential increase tied to the number of events and product expansion, requiring high-performance logging and query support.

DDL Statements - Create Commands:

1. Create the Venue table

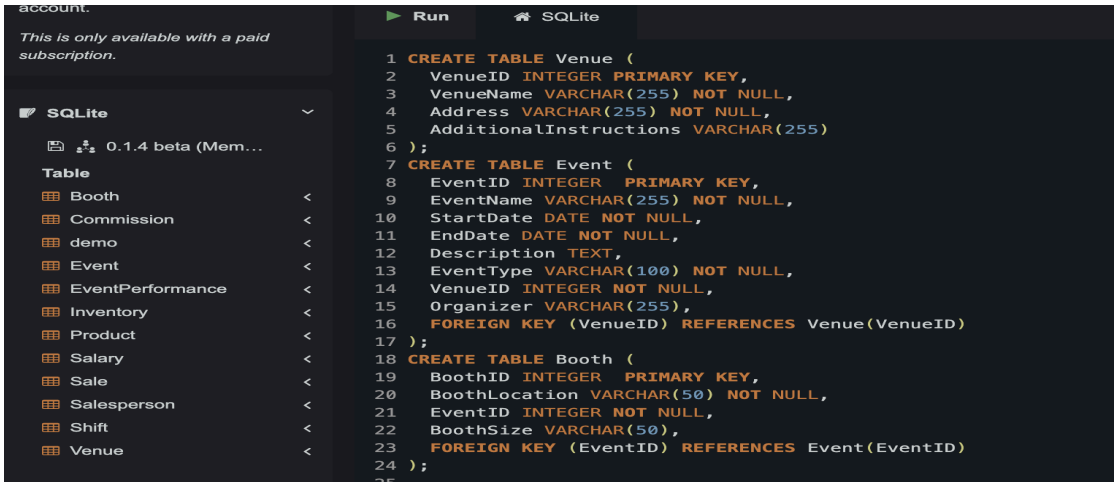
CREATE TABLE Venue (
VenueID INTEGER PRIMARY KEY,
VenueName VARCHAR(255) NOT NULL,
Address VARCHAR(255) NOT NULL,
AdditionalInstructions VARCHAR(255));

2. Create the Event table

CREATE TABLE Event (
EventID INTEGER PRIMARY KEY,
EventName VARCHAR(255) NOT NULL,
StartDate DATE NOT NULL,
EndDate DATE NOT NULL,
Description TEXT,
EventType VARCHAR(100) NOT NULL,
VenueID INTEGER NOT NULL,
Organizer VARCHAR(255),
FOREIGN KEY (VenueID) REFERENCES Venue(VenueID));

3. Create the Booth table

CREATE TABLE Booth (
BoothID INTEGER PRIMARY KEY,
BoothLocation VARCHAR(50) NOT NULL,
EventID INTEGER NOT NULL,
BoothSize VARCHAR(50),
FOREIGN KEY (EventID) REFERENCES Event(EventID));



1. Create the Product table

CREATE TABLE Product (
ProductID INTEGER PRIMARY KEY,
ProductName VARCHAR(255) NOT NULL,
WholesaleCost DECIMAL(10,2) NOT NULL,
MinimumSellingPrice DECIMAL(10,2) NOT NULL,
Description TEXT,
Category VARCHAR(100));

2. Create the Inventory table (for current stock tracking)

CREATE TABLE Inventory (
InventoryID INTEGER PRIMARY KEY,
ProductID INTEGER NOT NULL,
CurrentStock INTEGER NOT NULL,
ReorderLevel INTEGER NOT NULL,
LastRestockDate DATE,
FOREIGN KEY (ProductID) REFERENCES Product(ProductID));

Run SQLite

```sql
CREATE TABLE Product (
  ProductID INTEGER  PRIMARY KEY,
  ProductName VARCHAR(255) NOT NULL,
  WholesaleCost DECIMAL(10,2) NOT NULL,
  MinimumSellingPrice DECIMAL(10,2) NOT NULL,
  Description TEXT,
  Category VARCHAR(100)
);

CREATE TABLE Inventory (
  InventoryID INTEGER  PRIMARY KEY,
  ProductID INTEGER NOT NULL,
  CurrentStock INTEGER NOT NULL,
  ReorderLevel INTEGER NOT NULL,
  LastRestockDate DATE,
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

CREATE TABLE Salesperson (
  SalespersonID INTEGER  PRIMARY KEY,
  FirstName VARCHAR(100) NOT NULL,
  LastName VARCHAR(100) NOT NULL,
  Address VARCHAR(255),
  PhoneNumber VARCHAR(20),
  Email VARCHAR(100),
  HireDate DATE
);
```

3. Create the Salesperson table

```
CREATE TABLE Salesperson (
SalespersonID INTEGER  PRIMARY KEY,
FirstName VARCHAR(100) NOT NULL,
LastName VARCHAR(100) NOT NULL,
Address VARCHAR(255),
PhoneNumber VARCHAR(20),
Email VARCHAR(100),
HireDate DATE);
```

4. Create the Salary table for base salary details

```
CREATE TABLE Salary (
SalaryID INTEGER  PRIMARY KEY,
SalespersonID INTEGER NOT NULL,
BaseSalary DECIMAL(10,2) NOT NULL,
EffectiveDate DATE NOT NULL,
FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID));
```

5. Create the Shift table to capture salespersons assignments

```
CREATE TABLE Shift (
ShiftID INTEGER  PRIMARY KEY,
SalespersonID INTEGER NOT NULL,
BoothID INTEGER NOT NULL,
ShiftStart DATETIME NOT NULL,
ShiftEnd DATETIME NOT NULL,
Role VARCHAR(50),
FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID),
FOREIGN KEY (BoothID) REFERENCES Booth(BoothID));
```

```
Run        SQLite

CREATE TABLE Salesperson (
  SalespersonID INTEGER  PRIMARY KEY,
  FirstName VARCHAR(100) NOT NULL,
  LastName VARCHAR(100) NOT NULL,
  Address VARCHAR(255),
  PhoneNumber VARCHAR(20),
  Email VARCHAR(100),
  HireDate DATE
);
CREATE TABLE Salary (
  SalaryID INTEGER  PRIMARY KEY,
  SalespersonID INTEGER NOT NULL,
  BaseSalary DECIMAL(10,2) NOT NULL,
  EffectiveDate DATE NOT NULL,
  FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID)
);

CREATE TABLE Shift (
  ShiftID INTEGER  PRIMARY KEY,
  SalespersonID INTEGER NOT NULL,
  BoothID INTEGER NOT NULL,
  ShiftStart DATETIME NOT NULL,
  ShiftEnd DATETIME NOT NULL,
  Role VARCHAR(50),
  FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID),
  FOREIGN KEY (BoothID) REFERENCES Booth(BoothID)
);
```

6. Create the Sale table to record sales transactions

CREATE TABLE Sale (
SaleID INTEGER PRIMARY KEY,
SaleDateTime DATETIME NOT NULL,
QuantitySold INTEGER NOT NULL,
FinalSellingPrice DECIMAL(10,2) NOT NULL,
SalespersonID INTEGER NOT NULL, -- Who made the sale
ProductID INTEGER NOT NULL, -- Which product was sold
EventID INTEGER NOT NULL, -- At which event the sale occurred
BoothID INTEGER NOT NULL, -- Through which booth
PaymentMethod VARCHAR(50),
DiscountApplied DECIMAL(10,2),
FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID),
FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
FOREIGN KEY (EventID) REFERENCES Event(EventID),
FOREIGN KEY (BoothID) REFERENCES Booth(BoothID));

7. Create the Commission table to capture commission details per sale

CREATE TABLE Commission (
CommissionID INTEGER PRIMARY KEY,
SaleID INTEGER NOT NULL, -- Reference to the specific sale
SalespersonID INTEGER NOT NULL, -- Who earned the commission

CommissionRate DECIMAL(5,2) NOT NULL,
CommissionAmount DECIMAL(10,2) NOT NULL,
CommissionDate DATETIME NOT NULL,
CommissionMonth DATE NOT NULL,
FOREIGN KEY (SaleID) REFERENCES Sale(SaleID),
FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID));

```
Run        SQLite

CREATE TABLE Sale (
  SaleID INTEGER  PRIMARY KEY,
  SaleDateTime DATETIME NOT NULL,
  QuantitySold INTEGER NOT NULL,
  FinalSellingPrice DECIMAL(10,2) NOT NULL,
  SalespersonID INTEGER NOT NULL,
  ProductID INTEGER NOT NULL,
  EventID INTEGER NOT NULL,
  BoothID INTEGER NOT NULL,
  PaymentMethod VARCHAR(50),
  DiscountApplied DECIMAL(10,2),
  FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
  FOREIGN KEY (EventID) REFERENCES Event(EventID),
  FOREIGN KEY (BoothID) REFERENCES Booth(BoothID)
);
CREATE TABLE Commission (
  CommissionID INTEGER  PRIMARY KEY,
  SaleID INTEGER NOT NULL,
  SalespersonID INTEGER NOT NULL,
  CommissionRate DECIMAL(5,2) NOT NULL,
  CommissionAmount DECIMAL(10,2) NOT NULL,
  CommissionDate DATETIME NOT NULL,
  CommissionMonth DATE NOT NULL,
  FOREIGN KEY (SaleID) REFERENCES Sale(SaleID),
  FOREIGN KEY (SalespersonID) REFERENCES Salesperson(SalespersonID)
);
```

8. Create the EventPerformance table for performance metrics per event

CREATE TABLE EventPerformance (
PerformanceID INTEGER PRIMARY KEY,
EventID INTEGER NOT NULL,
TotalSales DECIMAL(10,2) NOT NULL,
TotalCustomers INTEGER NOT NULL,
Feedback TEXT,
FOREIGN KEY (EventID) REFERENCES Event(EventID));

```
Run        SQLite

CREATE TABLE EventPerformance (
  PerformanceID INTEGER  PRIMARY KEY,
  EventID INTEGER NOT NULL,
  TotalSales DECIMAL(10,2) NOT NULL,
  TotalCustomers INTEGER NOT NULL,
  Feedback TEXT,
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
```

DDL Statements - Insert Commands:

1. Venue

```
INSERT INTO Venue (VenueName, Address, AdditionalInstructions)
VALUES
('Downtown Expo Center', '123 Main St, Chicago, IL', 'Parking available behind the venue'),
('Riverside Park', '456 River Rd, Aurora, IL', 'Use north gate for entry'),
('Greenfield Grounds', '789 Green Blvd, Elgin, IL', 'Enter from south gate near parking'),
('Lakeside Pavilion', '101 Lakeview Dr, Naperville, IL', 'Shuttle service available from downtown'),
('West Loop Hall', '500 Madison St, Chicago, IL', 'Check-in at the east lobby');
```

2. Event

```
INSERT INTO Event (EventName, StartDate, EndDate, Description, EventType, VenueID, Organizer)
VALUES
('Summer Rib Fest', '2025-07-10', '2025-07-12', 'Annual rib cook-off and fair', 'Food Festival', 2, 'City Events Team'),
('Tech Gadget Expo', '2025-08-05', '2025-08-07', 'Showcase of latest gadgets and tech gear', 'Trade Show', 1, 'Innovate LLC'),
('Music in the Park', '2025-07-20', '2025-07-21', 'Live performances by local bands', 'Music Festival', 3, 'Harmony Org'),
('Green Market Days', '2025-07-15', '2025-07-16', 'Sustainable products and eco-friendly startups', 'Market Fair', 4, 'EcoLife Group'),
('Startup Pitch Fest', '2025-08-15', '2025-08-16', 'Startup founders present their ideas', 'Business Expo', 5, 'Venture Builders Inc');
```

3. Booth

```
INSERT INTO Booth (BoothLocation, EventID, BoothSize)
VALUES
('S11', 1, 'Large'),
('T03', 2, 'Medium'),
('B01', 3, 'Small'),
('C15', 3, 'Medium'),
('D22', 4, 'Large'),
('E09', 4, 'Small'),
('F14', 5, 'Medium'),
('G18', 5, 'Large'),
('H05', 2, 'Small'),
('I10', 1, 'Medium');
```

```
Run    SQLite

INSERT INTO Venue (VenueName, Address, AdditionalInstructions)
VALUES
('Downtown Expo Center', '123 Main St, Chicago, IL', 'Parking available behind the venue'),
('Riverside Park', '456 River Rd, Aurora, IL', 'Use north gate for entry'),
('Greenfield Grounds', '789 Green Blvd, Elgin, IL', 'Enter from south gate near parking'),
('Lakeside Pavilion', '101 Lakeview Dr, Naperville, IL', 'Shuttle service available from downtown')
('West Loop Hall', '500 Madison St, Chicago, IL', 'Check-in at the east lobby');

INSERT INTO Event (EventName, StartDate, EndDate, Description, EventType, VenueID, Organizer)
VALUES
('Summer Rib Fest', '2025-07-10', '2025-07-12', 'Annual rib cook-off and fair', 'Food Festival', 2,
('Tech Gadget Expo', '2025-08-05', '2025-08-07', 'Showcase of latest gadgets and tech gear', 'Trade
('Music in the Park', '2025-07-20', '2025-07-21', 'Live performances by local bands', 'Music Festiv
('Green Market Days', '2025-07-15', '2025-07-16', 'Sustainable products and eco-friendly startups',
('Startup Pitch Fest', '2025-08-15', '2025-08-16', 'Startup founders present their ideas', 'Busines

INSERT INTO Booth (BoothLocation, EventID, BoothSize)
VALUES
('S11', 1, 'Large'),
('T03', 2, 'Medium'),
('B01', 3, 'Small'),
('C15', 3, 'Medium'),
('D22', 4, 'Large'),
('E09', 4, 'Small'),
('F14', 5, 'Medium'),
('G18', 5, 'Large'),
('H05', 2, 'Small'),
('I10', 1, 'Medium');
```

DDL Statements -Update Commands

1. Update inventory stock after a sale

UPDATE Inventory
SET CurrentStock = CurrentStock - 2
WHERE ProductID = 1;

2. Update salesperson's phone number

UPDATE Salesperson
SET PhoneNumber = '312-555-9999'
WHERE SalespersonID = 1;

3. Update the final selling price for a sale

UPDATE Sale
SET FinalSellingPrice = 22.00, DiscountApplied = 2.00
WHERE SaleID = 1;

4. Update event feedback after customer survey

UPDATE EventPerformance
SET Feedback = 'Excellent food options and entertainment.'

WHERE EventID = 1;

Run SQLite

```sql
UPDATE Inventory
SET CurrentStock = CurrentStock - 2
WHERE ProductID = 1;

UPDATE Salesperson
SET PhoneNumber = '312-555-9999'
WHERE SalespersonID = 1;

UPDATE Sale
SET FinalSellingPrice = 22.00, DiscountApplied = 2.00
WHERE SaleID = 1;

UPDATE EventPerformance
SET Feedback = 'Excellent food options and entertainment.'
WHERE EventID = 1;
```

DDL Statements - Delete Statements:

- Delete statements to delete an Event and cascading relations

DELETE FROM Commission
WHERE SaleID IN (SELECT SaleID FROM Sale WHERE EventID = 1);

DELETE FROM Sale
WHERE EventID = 1;

DELETE FROM Shift
WHERE BoothID IN (SELECT BoothID FROM Booth WHERE EventID = 1);

DELETE FROM Booth
WHERE EventID = 1;

DELETE FROM EventPerformance
WHERE EventID = 1;

DELETE FROM Event
WHERE EventID = 1;

```sql
DELETE FROM Commission
WHERE SaleID IN (SELECT SaleID FROM Sale WHERE EventID = 1);

DELETE FROM Sale
WHERE EventID = 1;

DELETE FROM Shift
WHERE BoothID IN (SELECT BoothID FROM Booth WHERE EventID = 1);

DELETE FROM Booth
WHERE EventID = 1;

DELETE FROM EventPerformance
WHERE EventID = 1;

DELETE FROM Event
WHERE EventID = 1;
```

DDL Statements - Select Queries:

- **Total Sales and Commission Earned by Each Salesperson (Per Event)**

```sql
SELECT
sp.SalespersonID,
CONCAT(sp.FirstName, ' ', sp.LastName) AS SalespersonName,
e.EventName,
SUM(s.FinalSellingPrice) AS TotalSales,
SUM(c.CommissionAmount) AS TotalCommission
FROM Sale s
JOIN Salesperson sp ON s.SalespersonID = sp.SalespersonID
JOIN Event e ON s.EventID = e.EventID
LEFT JOIN Commission c ON s.SaleID = c.SaleID
GROUP BY sp.SalespersonID, sp.FirstName, sp.LastName, e.EventName;
```

```sql
SELECT
    sp.SalespersonID,
    CONCAT(sp.FirstName, ' ', sp.LastName) AS SalespersonName,
    e.EventName,
    SUM(s.FinalSellingPrice) AS TotalSales,
    SUM(c.CommissionAmount) AS TotalCommission
FROM Sale s
JOIN Salesperson sp ON s.SalespersonID = sp.SalespersonID
JOIN Event e ON s.EventID = e.EventID
LEFT JOIN Commission c ON s.SaleID = c.SaleID
GROUP BY sp.SalespersonID, sp.FirstName, sp.LastName, e.EventName;
```

SalespersonID	SalespersonName	EventName	TotalSales	TotalCommission
1	Alice Johnson	Summer Rib Fest	80	8
2	Mark Lee	Tech Gadget Expo	160	16
3	Nina Patel	Music in the Park	25	2
4	Jacob Wright	Music in the Park	50	5
5	Linda Garcia	Green Market Days	20	2
6	Tom Anderson	Green Market Days	45	4.5
7	Priya Mehta	Startup Pitch Fest	75	9
8	Samuel Baker	Startup Pitch Fest	90	9

- **Current Inventory Levels and Reorder Suggestions**

```sql
SELECT
p.ProductName,
i.CurrentStock,
i.ReorderLevel,
CASE
WHEN i.CurrentStock <= i.ReorderLevel THEN 'Reorder Needed'
ELSE 'Stock OK'
END AS StockStatus
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID;
```

```sql
SELECT
    p.ProductName,
    i.CurrentStock,
    i.ReorderLevel,
    CASE
        WHEN i.CurrentStock <= i.ReorderLevel THEN 'Reorder Needed'
        ELSE 'Stock OK'
    END AS StockStatus
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID;
```

ProductName	CurrentStock	ReorderLevel	StockStatus
CoolFan Pro	100	20	Stock OK
Mini Drone X2	40	15	Stock OK
Bubble Blaster	200	50	Stock OK
ChillBuddy Neck Fan	75	25	Stock OK
LED Spinner	120	30	Stock OK
RC Speed Car	90	20	Stock OK
Pop Rocket Launcher	60	15	Stock OK
SmartTemp Bottle	150	40	Stock OK
Misting Water Bottle	110	35	Stock OK

- **Event-wise Revenue and Attendance Report**

```sql
SELECT
    e.EventName,
    e.StartDate,
    ep.TotalSales,
    ep.TotalCustomers,
    ep.Feedback
FROM Event e
JOIN EventPerformance ep ON e.EventID = ep.EventID;
```

```sql
SELECT
    e.EventName,
    e.StartDate,
    ep.TotalSales,
    ep.TotalCustomers,
    ep.Feedback
FROM Event e
JOIN EventPerformance ep ON e.EventID = ep.EventID;
```

EventName	StartDate	TotalSales	TotalCustomers	Feedback
Summer Rib Fest	2025-07-10	1200	450	Well-attended, great energy
Tech Gadget Expo	2025-08-05	1800	300	Moderate turnout, very eng
Music in the Park	2025-07-20	950	380	Positive vibe, music apprec
Green Market Days	2025-07-15	800	290	Eco-conscious crowd, slow
Startup Pitch Fest	2025-08-15	1250	320	High interest from investors

- **Monthly Commission Summary by Salesperson**

```sql
SELECT
   sp.SalespersonID,
   sp.FirstName || ' ' || sp.LastName AS SalespersonName,
   strftime('%Y-%m', c.CommissionMonth) AS CommissionMonth,
   SUM(c.CommissionAmount) AS TotalCommission
FROM Commission c
JOIN Salesperson sp ON c.SalespersonID = sp.SalespersonID
GROUP BY sp.SalespersonID, strftime('%Y-%m', c.CommissionMonth)
ORDER BY CommissionMonth DESC;
```

```sql
SELECT
    sp.SalespersonID,
    sp.FirstName || ' ' || sp.LastName AS SalespersonName,
    strftime('%Y-%m', c.CommissionMonth) AS CommissionMonth,
    SUM(c.CommissionAmount) AS TotalCommission
FROM Commission c
JOIN Salesperson sp ON c.SalespersonID = sp.SalespersonID
GROUP BY sp.SalespersonID, strftime('%Y-%m', c.CommissionMonth)
ORDER BY CommissionMonth DESC;
```

SalespersonID	SalespersonName	CommissionMonth	TotalCommission
2	Mark Lee	2025-08	16
7	Priya Mehta	2025-08	9
8	Samuel Baker	2025-08	9
1	Alice Johnson	2025-07	8
3	Nina Patel	2025-07	2
4	Jacob Wright	2025-07	5
5	Linda Garcia	2025-07	2
6	Tom Anderson	2025-07	4.5

- **Salesperson Shift Schedule for a Specific Event**

```
SELECT
    e.EventName,
    b.BoothLocation,
    sp.FirstName,
    sp.LastName,
    sh.ShiftStart,
    sh.ShiftEnd,
    sh.Role
FROM Shift sh
JOIN Booth b ON sh.BoothID = b.BoothID
JOIN Event e ON b.EventID = e.EventID
JOIN Salesperson sp ON sh.SalespersonID = sp.SalespersonID
WHERE e.EventName = 'Summer Rib Fest';
```

```
SELECT
    e.EventName,
    b.BoothLocation,
    sp.FirstName,
    sp.LastName,
    sh.ShiftStart,
    sh.ShiftEnd,
    sh.Role
FROM Shift sh
JOIN Booth b ON sh.BoothID = b.BoothID
JOIN Event e ON b.EventID = e.EventID
JOIN Salesperson sp ON sh.SalespersonID = sp.SalespersonID
WHERE e.EventName = 'Summer Rib Fest';
```

EventName	BoothLocation	FirstName	LastName	ShiftStart	ShiftEnd	Role
Summer Rib F...	S11	Alice	Johnson	2025-07-10 09...	2025-07-10 17...	Lead
Summer Rib F...	S11	Priya	Mehta	2025-07-16 10...	2025-07-16 17...	Assistant
Summer Rib F...	I10	Benjamin	Scott	2025-07-25 09...	2025-07-25 17...	Lead
Summer Rib F...	S11	Tina	Chowdhury	2025-07-26 10...	2025-07-26 18...	Assistant

- **Top Selling Products per EventBooth**

```
SELECT
    e.EventName,
    p.ProductName,
    SUM(s.QuantitySold) AS TotalUnitsSold,
    SUM(s.FinalSellingPrice) AS RevenueGenerated
FROM Sale s
JOIN Product p ON s.ProductID = p.ProductID
JOIN Event e ON s.EventID = e.EventID
GROUP BY e.EventName, p.ProductName
ORDER BY RevenueGenerated DESC;
```

```
SELECT
    e.EventName,
    p.ProductName,
    SUM(s.QuantitySold) AS TotalUnitsSold,
    SUM(s.FinalSellingPrice) AS RevenueGenerated
FROM Sale s
JOIN Product p ON s.ProductID = p.ProductID
JOIN Event e ON s.EventID = e.EventID
GROUP BY e.EventName, p.ProductName
ORDER BY RevenueGenerated DESC;
```

EventName	ProductName	TotalUnitsSold	RevenueGenerated
Tech Gadget Expo	Kiddie Bubble Gun	2	100
Startup Pitch Fest	Misting Water Bottle	3	90
Startup Pitch Fest	SmartTemp Bottle	1	75
Tech Gadget Expo	Mini Drone X2	1	60
Music in the Park	LED Spinner	2	50
Summer Rib Fest	CoolFan Pro	2	50
Green Market Days	Pop Rocket Launcher	2	45
Summer Rib Fest	Bubble Blaster	3	30

Conclusion:

This report presents an end-to-end design of the Foxcore Retail database, developed by following the supplied Entity-Relationship (ER) diagram and schema specifications. The proposed solution effectively addresses the company's requirement for an overall system for managing events, sales, sales commission, products, events, and event performance.

Foxcore Retail has transitioned from its manual spreadsheet-driven systems to an integrated, fully relational database to overcome data management challenges such as redundancy, inconsistency, and administrative overhead. The new database offers real-time data access, role-based security controls, and embedded reporting for discrete business functions. These capabilities increase operational efficiency daily and empower management with timely insights to drive strategic decision-making.

Moreover, the system is built to be scalable and adaptable, ensuring the growth of Foxcore Retail adding more products, expanding more venues, and hiring more sales consultants. The database can easily accommodate this growth without impacting current performance.

Ultimately, this database positions Foxcore Retail up for long-term success by strengthening operational processes and making it easier to tell our data story about which resources are being used for what, where, why those decisions are being made, and how they will inform strategic growth.